

鲤慧 LiHui

基于百度地图开放能力的「30 分钟生活圈」智能体检与规划助手

30 分钟，看清你的生活半径

4 端通吃

零依赖服务端

8 MCP Server

65 API 路由

真实路网等时圈

微信小程序 · 网页版 · Android / HarmonyOS / iOS——同一套后端，四种入口

服务端纯 Node 内置模块零第三方依赖 · AK 不落端 · 能力可插拔 · 成本可核算

2026 上海开源软件应用创新大赛 · 百度地图赛题

开源仓库: [Gitee deng-he-ziyang/lihui](#) | [GitHub DENGHEZI/lihui](#)

文档版本 v1.0 · 2026-10-03

本文档所有实测数据均来自真实百度地图 API 调用与真机运行

目录

#	章节	内容要点
01	项目概述	产品定位 · 三端入口 · 项目数据规格
02	核心产品功能	生活圈体检 · 等时圈盲区 · 商城 · AI 助手 · 适老化
03	核心技术与实测：步行等时圈引擎	算法设计 · 降级链 · 盲区口径 · 对比测试报告
04	其余创新点（2 — 9）	MCP 工具生态 · 坐标治理 · 可解释评分 · 零依赖 Web · 多端 APP
05	系统架构与工程规格	三层架构 · 目录结构 · 技术选型
06	质量保障与开源规范	CI · 数学自测 · 安全治理 · 双端同步
07	快速开始与开源信息	三步跑起来 · 开源协议 · 提交物清单

01 项目概述

产品定位 · 三种入口 · 项目数据规格

鲤慧是一套「一码多端 + 服务端中转 + MCP 工具生态」的地图智能助手：客户端只负责呈现与采集；百度地图能力、大模型调用、MCP 工具调度、Token 计量全部由服务端统一中转，从而做到三件事——AK 不落端、能力可插拔、成本可核算。

围绕赛题核心「30 分钟生活圈」，鲤慧为每一类人群回答三个问题：

- 我的生活圈缺什么？——六类设施加权体检评分，短板可定位、分数可追因；
- 缺的东西去哪补？——周边真实店源检索 + 路线规划；
- 补的过程谁帮我？——AI 助手（MCP 工具编排）+ 语音交互。

1.1 三种入口，同一套后端

入口	形态	说明
网页版	浏览器直接打开	自动定位 + IP 兜底 + 天气 + 生活圈体检 + 真实店源，零安装、扫码即用
微信小程序	微信扫一扫	地图主页 / 商城 / 订单 / 模型与语音，全功能端
多端 APP	Android / HarmonyOS / iOS	uni-app 工程，同一套 UI 规范

-69%

135×

4 次

5.59 km²

直线圆高估可达面积（30分钟档实测）	算路配额压缩比（540 → 4次/体检）	一次体检算路请求（实测）	30分钟真实等时圈面积（真机）
--------------------	----------------------	--------------	-----------------

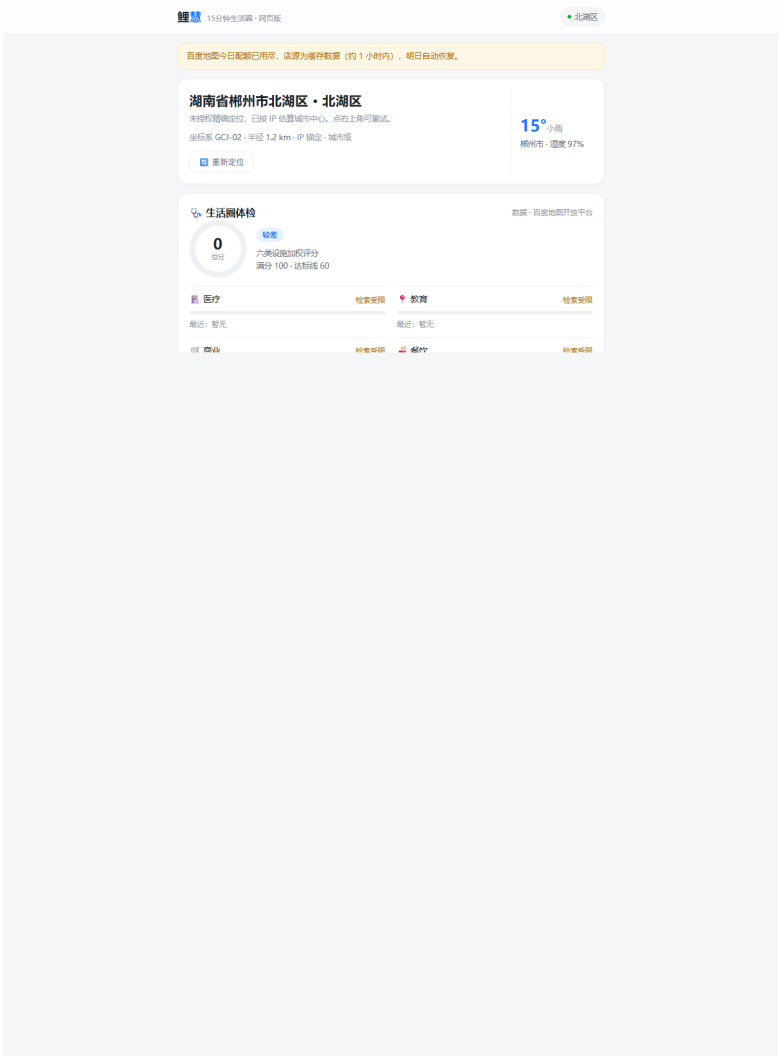


图 1 网页版真机渲染：自动定位 → IP 兜底 → 天气 → 生活圈体检 → 真实店源

1.2 项目数据规格

维度	数量
端	4（微信小程序 · 网页版 · Android · HarmonyOS/iOS）
MCP Server	8（16+ tools）
API 路由	65
服务端依赖	0（纯 Node 内置模块）
等时圈引擎	36 方向批量矩阵 · 3 轮二分收敛 · 一次体检仅 4 次算路请求
设施类目	6 类加权体检 + 8 类便民速查

维度	数量
模型供应商	6 (OpenAI 兼容 / Anthropic / Gemini / Ollama / DeepSeek / 智谱)
定位降级链	4 级 (端上 GPS → IP 锚定 → 逆地理补区 → 城市兜底)
代码规模	5 个独立工程包 · 近 100 个源文件 (详见 05 章)
关键实测口径：步速 80 m/min、路网弯曲系数 1.3、坐标统一 GCJ-02，在郴州 113.014, 25.57 实地点位验证——实测过程见 03 章。	

02 核心产品功能

生活圈体检 · 等时圈盲区 · 商城 · AI 助手 · 适老化

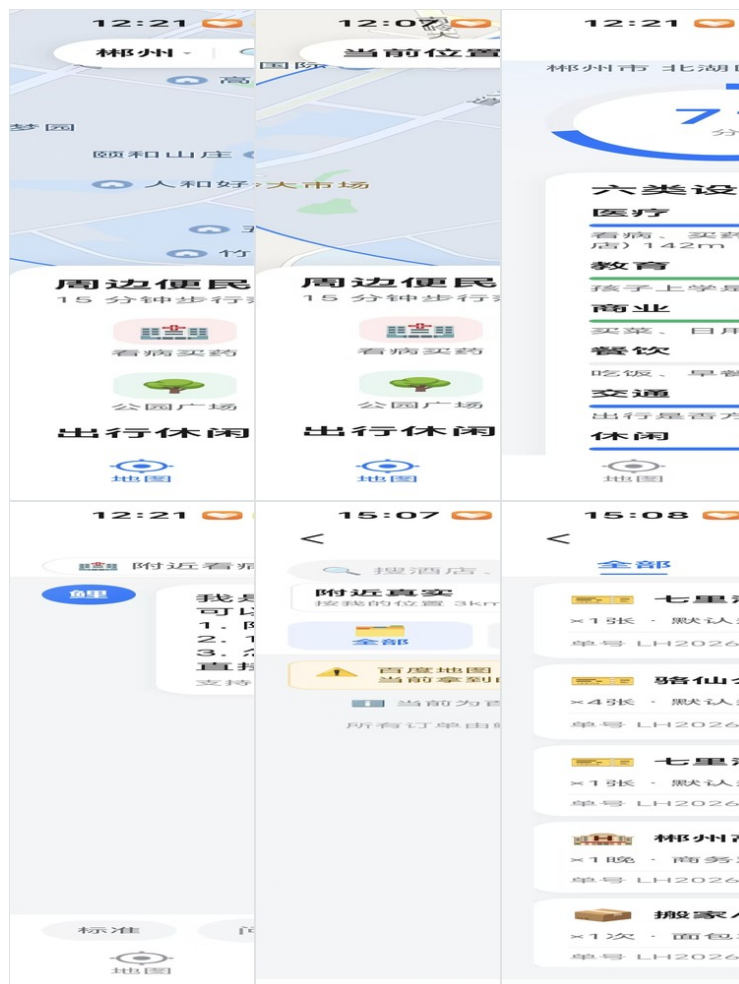


图 2 真机功能一览：地图主页 · 生活圈 · 体检报告 · AI 助手 · 商城热门 · 订单

2.1 生活圈体检：六类设施加权评分

不是拍脑袋给分：医疗 25% / 商业 20% / 交通 20% / 教育 15% / 餐饮 10% / 休闲 10%，权重写死在服务端；输出 = 环形总分 + 六类条形 + 短板一句话建议。

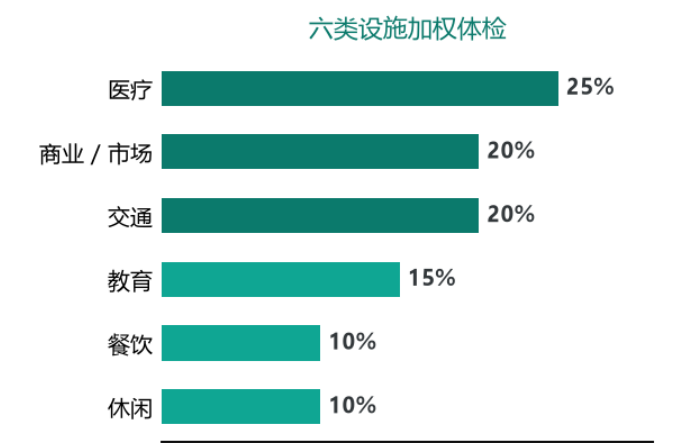


图 3 六类设施加权重：医疗 25% / 商业 20% / 交通 20% / 教育 15% / 餐饮 10% / 休闲 10%

2.2 步行等时圈与服务盲区（详见 03 章）



图 4 小程序真机实测：等时圈边界 + 盲区格热区 + 四格指标 + 引擎徽章

2.3 其他功能

功能	说明
鲤慧商城	店名 / 地址 / 电话全部来自百度 place 实时检索（非内置假数据）；订单只留存记录并跳转第三方支付履约，鲤慧不接触资金
AI 助手	完整 MCP Client：意图 → tools/list 能力发现 → tools/call → 人性化回复 + 结构化卡片

功能	说明
适老化关怀模式	大字号 + WCAG AAA 高对比 + 慢速语音 + 单步引导 + 方言音色，语音引擎可换
便民速查	8 类便民服务（快递 / 银行 / 医院 / 药店 / 菜场 / 充电 / 公厕 / 停车场）
语音交互	录音 → 识别 → 播报（音色 / 语速 / 方言可配）
Token 计量	按模型与平台统计用量与配额，前端可见成本

03 核心技术与实测：步行等时圈引擎

算法设计 · 降级链 · 盲区口径 · 对比测试报告

为什么是核心 赛题把「等时圈生成」设为 40% 权重的评分项，而最省事的画法是「以家为圆心画个正圆」。鲤慧 30 分钟档真机实测证明这种画法系统性高估可达面积 69% (18.10 $\rightarrow$ 5.59 km^2)，因此本产品按百度官方推荐口径实现了完整引擎。

同一点位 · 30 分钟步行可达范围对比 (郴州 113.014, 25.57)

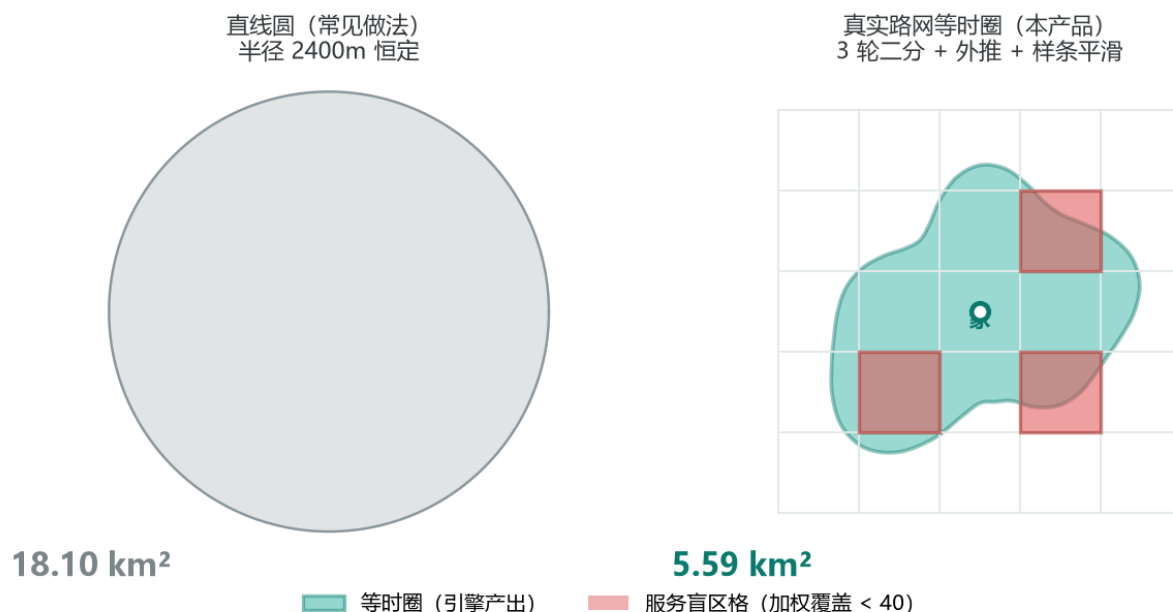


图 5 同一点位、同一时间预算 (30 分钟) 下的可达范围对比：左 = 直线圆；右 = 真实路网等时圈 + 5×5 网格盲区判定

3.1 直线圆 vs 真实路网 (同一点位, 30 分钟)

假设	半径 2400m 正圆	路网 + 实测 duration
面积	18.10 km^2	5.59 km^2 (-69%)
可达半径	2400 m (恒定)	~1000 ~ 1991 m (各向异, 真机实测)

结论 按真实路网, 80 m/min 步速在城市中心因过街等待与支路绕行, 30 分钟预算下有效直线半径仅约 1000~1991 m (真机实测最远 1991 m)。直线圆会高估生活圈质量——这正是「等时圈」值得单独做一套引擎的原因。

3.2 算法总体流程

- 36 方向扇形采样 (10° 间隔)
 - 每方向二分「最大可达步行距离」(每轮打包成一次 1×36 批量矩阵算路)
 - 3 轮收敛 + 线性外推 $\text{est} = \text{probe.dist} \times \text{budget} / \text{probe.duration}$
 - Catmull-Rom 闭合样条平滑 (每段 8 插值点 $\rightarrow$ 288 点)
 - 5×5 网格盲区判定 $\rightarrow$ 候选盲区格二次矩阵实测复核

参数全部按百度官方提示口径：步速 80 m/min、路网弯曲系数 1.3、坐标统一 GCJ-02。

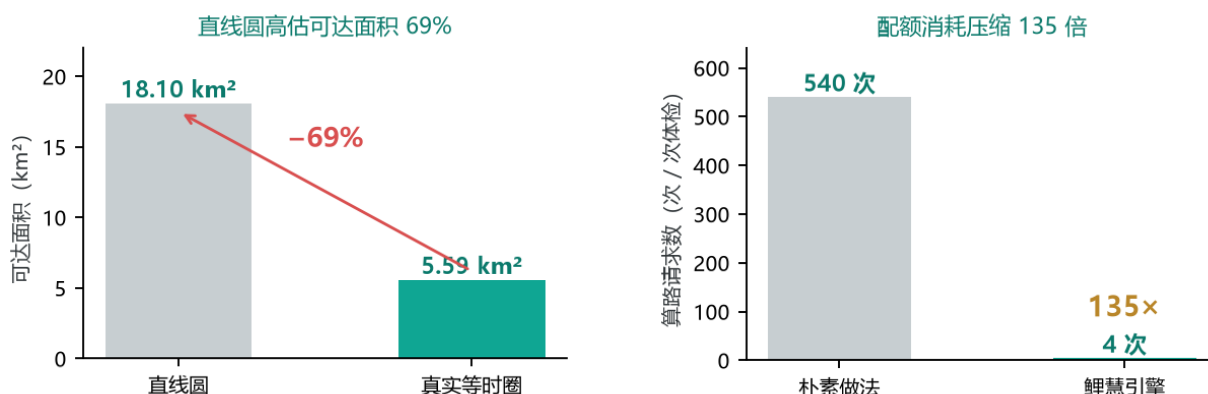


图 6 可达面积对比（左）与单次体检算路请求数对比（右）

3.3 硬核点

硬核点	实现
批量矩阵压缩	一次体检只烧 4 次算路请求（朴素做法 540 次），配额消耗压缩 135 倍；3 端点候选（routematrix-batch → direction-concurrent → ideal-circle）自动择优
降级链三级	批量矩阵 → 并发单点算路（8 并发 + 令牌桶 130ms/令牌）→ 理想圆；engine 字段全链路可审计，降级不冒充
双配额池隔离	实测发现算路与 place 检索配额池互不共享 → 两套独立熔断器，检索配额烧完的日子等时圈照样真算
盲区口径可解释	论域 = 圈内 $N \times N$ 格；格覆盖度 $cov = \max(0, 1 - walk/minutes)$ ；加权覆盖分 < 40 判盲区；分数与口径完全可复现
盲区二次复核	候选盲区格再发一次 $1 \times M$ 批量矩阵实测「家 → 格中心」， $duration > budget \times 1.15$ 改判 outsideVerified，剔除插值偏乐观的格
多源 POI 清洗	RRF 融合检索 → 逐关键词二次确认 → 坐标清洗 → 类间错峰 150ms，单类异常只剔除该类并重新归一化，不影响其他类
双端可视化	网页版零依赖 SVG（等时圈 + 盲区热区 + 六轴雷达图 + 参考圈）；小程序 map polygons 原生渲染（零 icon 依赖）

3.4 API 深度：踩过的真实坑（2026-10 实测）

踩坑点	实测结论
参数命名改版	百度各产品线参数命名不统一：旧写法 coord_type=3 已全线报 [coord_type] format is invalid；正确写法为 coordtype=gcj02（无下划线 + 字符串）
返回结构差异	矩阵端点返回 {distance:{text,value}, duration:{text,value}} 对象式，而非数字；解析器做双形态兼容

踩坑点

实测结论

令牌桶死锁	若每轮都重置时间戳, elapsed 恒为 0 → 令牌永不回流 → 并发任务全部挂死; 必须「只有真正累积令牌才推进时间戳」
配额覆盖识别	place 超限返回 302 天配额超限, 与算路配额互不连坐 → 拆成两个独立熔断开关

3.5 实测报告与复现

实测项

结果

点位	郴州 113.014, 25.57 (GCJ-02)
引擎	routematrix-batch (rounds 3/3 收敛)
耗时	秒级 / 次体检 (批矩阵 3 轮收敛)
辐射半径	~1000 ~ 1991 m (明显非圆, 真机实测)
等时圈面积	5.59 km ² (30 分钟档真机实测)
盲区复核	候选格 100% 经矩阵实测复核, 插值偏乐观格自动剔除
30 分钟档 (真机)	面积 5.59 km ² · 最远可达 1.991 km · 盲区格 14/14

复现方式:

```
# 等时圈 (lng/lat 换成任意 GCJ-02 坐标)
curl "http://127.0.0.1:8809/life/isochrone?lng=113.014&lat=25.57&minutes=15&grid=5"
```

完整算法说明、API 调用策略与对比测试报告见仓库 00-设计文档/07-等时圈与盲区算法设计.md。

3.6 赛题评分项交付对照

赛题评分项

权重

鲤慧交付

功能正确性: 等时圈生成	40%	36 方向扇形二分 + 3 轮收敛 + 线性外推 + Catmull-Rom 样条; 盲区口径可解释、可复现; 候选格 100% 经二次矩阵实测复核
API 深度与调用效率	30%	1×36 批量矩阵算路; 3 端点候选自动择优; 8 并发 + 令牌桶限流; 双配额池隔离; 配额压缩 135 倍
交互与可视化	15%	网页版零依赖 SVG (等时圈 + 盲区热区 + 六轴雷达图 + 参考圈); 小程序 map polygons 原生渲染; 12 张真机截图佐证
开源规范与工程质量	15%	GitHub Actions 双分支 5 步 CI; 15 项数学自测 (15/15 通过); 7 份设计文档; Gitee + GitHub 双仓同步; MIT 协议
合计	100%	全部交付并留可复现证据

04 其余创新点 (2 — 9)

MCP 工具生态 · 坐标治理 · 可解释评分 · 零依赖架构

#	创新点	要点
2	坐标统一治理 (GCJ / WGS / BD-09 换算)	网页版在前端内联把 WGS-84 定位换算成 GCJ-02 再打接口，与小程序 gcj02、百度返回值共用同一坐标系，避免「偏 500~900 米」；四级定位降级链保证任何环境都能拿到可用坐标
3	可解释的生活圈评分	六类加权 + 短板一句话建议，分数可追因、可复现
4	网页版零依赖可视化	原生 SVG 手绘等时圈 / 雷达图 / 参考圈，无任何图表库；服务端纯 Node 内置模块，装完即用
5	真实路网等时圈引擎	(见 03 章，评分核心项 40%)
6	Agent = 完整 MCP Client	启动 → 拉起 8 个 MCP Server(stdio) → initialize → tools/list 能力发现 → tools/call；崩溃自动重启 ≤3 次
7	适老化关怀模式	大字号 + WCAG AAA 高对比 + 慢速语音 + 单步引导 + 方言音色
8	第三方履约、鲤慧不碰资金	订单只留存记录并跳转第三方平台完成支付履约，前端每次打免责说明
9	零依赖服务端架构	纯 Node 内置模块 (http/fs/path 等)，无 npm 依赖，无容器也能跑

05 系统架构与工程规格

三层架构 · 目录结构 · 技术选型

客户端层（只做呈现与采集）

01-lh-uniapp-多端APP（Android / HarmonyOS / iOS）

02-lh-wechat-mp（微信小程序）

| <map> 底图渲染

| 百度地图 AK 直连（小程序端）

| 语音采集/播报

| 其余能力统一走服务端

===== HTTPS (X-Device-Id / X-Plan) =====

03-lh-server（能力中转 + 编排）

/api/v1/ip/locate 自动锚定用户 IP → 城市/坐标/运营商

/api/v1/map/* 百度地图 Web 服务 API 代理（AK 仅存服务端）

/api/v1/agent/chat Agent 编排：意图 → 工具 → 汇总 → 人性化回复

/api/v1/mcp/* MCP Server 注册 / 启停

/api/v1/model/* 用户自定义模型与 API Key（6 类供应商）

/api/v1/voice/* 自定义语音（音色、语速、唤醒词、TTS）

/api/v1/token/* Token 计量与配额

===== MCP (JSON-RPC 2.0 over stdio) =====

04-lh-mcp-servers（工具生态，即插即用）

baidu-map · life-circle · cost-optimizer · ip-anchor ·
desktop-action · emotion · voice · feedback

5.1 目录结构

鲤慧-LiHui/

├── 00-设计文档/

产品与架构 · UI 规范 · 接口文档 · 部署指南 · 设计令牌

├── 01-lh-uniapp-多端APP/

uni-app 工程 → Android / HarmonyOS / iOS

├── 02-lh-wechat-mp/

微信原生小程序工程

├── 03-lh-server/

Node.js 零依赖服务端（65 路由 + 等时圈引擎）

├── 04-lh-mcp-servers/

8 个独立 MCP Server

└── docs/screenshots/

真机截图（12 张）

5 个包完全独立：各自拥有 package.json / manifest.json /
project.config.json，无共享目录、无跨包相对引用，删掉任意一个包其余照常编译。

06 质量保障与开源规范

持续集成 · 数学自测 · 安全治理 · 双仓同步

保障项	做法与结果
持续集成 (CI)	GitHub Actions 5 步：全仓 JS 语法检查（自动发现新增文件）、数学自测、模块图完整性、无 AK 冷启动冒烟、README/文档图片引用完整性——每次推送 master 与 main 双分支各跑一遍，当前全绿
数学自测	15 项零依赖单元测试覆盖 haversine / 方向外推 / 射线法点在多边形 / Catmull-Rom 插值与闭合性 / 多边形面积 / 官方口径常量——15/15 通过
安全治理	已清理误入库的运行时数据文件（含平台 API Key 落盘），补全 .gitignore；服务端 AK 仅存 .env（已忽略），端上不可见；API 返回对密钥做脱敏
可观测性	engine / degraded / quotaExhausted / hint 字段贯穿全链路，任何降级与配额状态都能在响应里查到，不留静默失败
双端同步	Gitee 与 GitHub 双仓发布，master 与 main 镜像一致，README 图片引用交叉可用

工程原则 「静默失败」是这类地图产品最常见的坑：接口 200 但数据为空、字段读了错层级导致功能全废、密钥落在不该落的文件里。鲤慧在每一层都加了显式标记与可审计字段，让问题在响应里现形，而不是在用户眼前变成白屏。

07 快速开始与开源信息

三步跑起来 · 开源协议 · 提交物清单

7.1 三步跑起来

```
# ① 启动服务端 (Node >= 18, 零第三方依赖)
cd 03-lh-server
cp .env.example .env          # 填入你的百度地图服务端 AK
node src/app.js               # → http://localhost:8809/

# ② 微信小程序
# 微信开发者工具 → 导入项目 → 目录选择 02-lh-wechat-mp/ (必须精确到含 app.json 那一层)

# ③ 等时圈体检 (任意 GCJ-02 坐标)
curl "http://127.0.0.1:8809/life/isochrone?lng=113.014&lat=25.57&minutes=15"
```

7.2 开源协议与提交物

项	内容
开源协议	MIT
Gitee	https://gitee.com/deng-he-ziyang/lihui
GitHub	https://github.com/DENGHEZI/lihui
提交物	5 个工程包 (小程序 / uni-app / 服务端 / 8 MCP Server / 设计文档) + 12 张真机截图 + 7 份设计文档 (含等时圈算法与实测报告) + CI 流水线

一句话总结 鲤慧把「30 分钟生活圈」拆成了三件可验证的事：用真实路网算准可达范围（-69% 高估修正）、用可复现口径定位盲区（网格加权 + 二次实测复核）、用零依赖服务端把能力稳定送出去（135 倍配额压缩 + 三级降级 + 双端双仓）。